R workshop part 3 - Data transformation

Jan Gačnik

Prepared: 2026-10-26

Data transformation in R

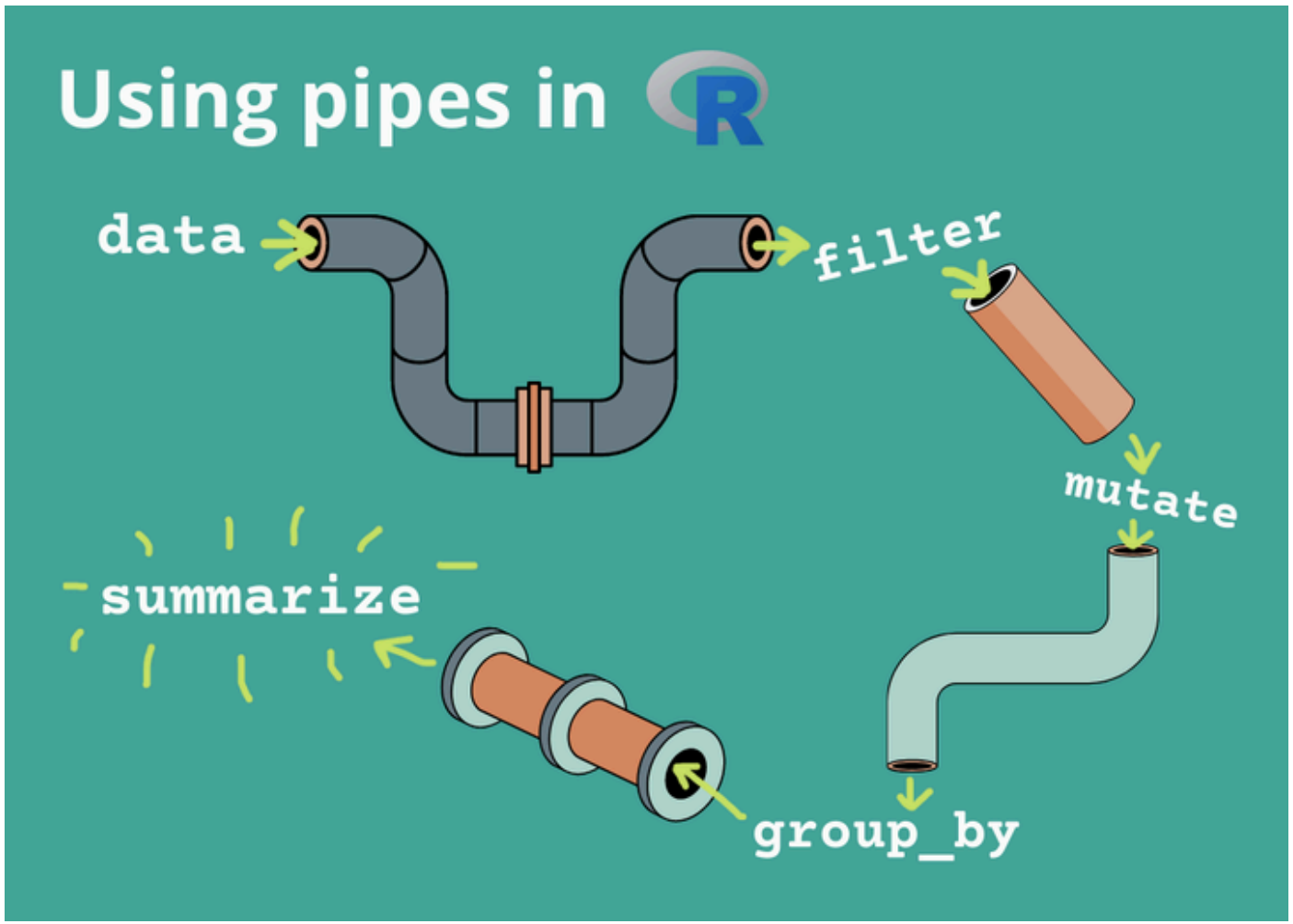
Data visualization is a great way to explore data, but the data is rarely in exact form as needed for making graphs.

In this presentation, we will learn how to:

- Filter the data
- Transform existing columns and create new ones
- Group data and make grouped summary statistics
- How to make data pipelines

Data transformation in R

Illustration of the data transformation process



Source: Celina Cheng, *R for Ecology*

Data transformation in R

We are staying in the `tidyverse` environment using the sub-package

`dplyr` 

Benefits:

- Human-readable syntax using predictable English verbs
- Intuitive piping, code as a sequence of steps
- Option to translate dplyr syntax to SQL query for databases

Cons:

- Slower than some other packages, can't handle giant data (>1 million rows)
- Functions keep improving and updating - old code may not work

Data transformation in R

Broadly, first functions studied today can be classified into these families:

- Functions that operate on **rows** of a data frame
- Functions that operate on **columns** of a data frame

1) Functions that operate on ROWS

`filter()`

Subsets a data frame, retaining all rows that satisfy your conditions.

First argument: data frame, second argument: logical test

```
1  penguins_filtered <- filter(penguins, island == "Dream")
```

Logical operator	Meaning
`<`	Less than
`<=`	Less than or equal to
`>`	More than
`>=`	More than or equal to
`==`	Equal to
`!=`	Not equal to
`A | B`	A or B
`A & B`	A and B

`filter()`

Subsets a data frame, retaining all rows that satisfy your conditions.

First argument: data frame, second argument: logical test

```
penguins_filtered <- filter(penguins, island == "Dream")
penguins_filtered_2 <- filter(penguins, flipper_len >= 190)
```

Logical operator	Meaning
`<`	Less than
`<=`	Less than or equal to
`>`	More than
`>=`	More than or equal to
`==`	Equal to
`!=`	Not equal to
`A | B`	A or B
`A & B`	A and B

`filter()`

Subsets a data frame, retaining all rows that satisfy your conditions.

First argument: data frame, second argument: logical test

```
penguins_filtered <- filter(penguins, island == "Dream")
penguins_filtered_2 <- filter(penguins, flipper_len >= 190)
penguins_filtered_3 <- filter(penguins, island == "Dream" & flipper_len >= 190)
```

Logical operator	Meaning
`<`	Less than
`<=`	Less than or equal to
`>`	More than
`>=`	More than or equal to
`==`	Equal to
`!=`	Not equal to
`A | B`	A or B
`A & B`	A and B

`filter()`

Subsets a data frame, retaining all rows that satisfy your conditions.

First argument: data frame, second argument: logical test

- Writing many `|` and `==` can get tedious:

```
1 penguins_filtered_4 <- filter(penguins, year == 2007 | year == 2008)
```

- Useful shortcut to do the same as above: `%in%`

```
1 penguins_filtered_4 <- filter(penguins, year %in% c(2007, 2008))
```

`filter()`

Subsets a data frame, retaining all rows that satisfy your conditions.

First argument: data frame, second argument: logical test

Common mistakes:

- Using `=` instead of `==`:

```
1 filter(penguins, year = 2007)
```

```
Error in `filter()`:
! We detected a named input.
ℹ This usually means that you've used `=` instead of `==`.
ℹ Did you mean `year == 2007`?
```

`filter()`

Subsets a data frame, retaining all rows that satisfy your conditions.

First argument: data frame, second argument: logical test

Common mistakes:

- Using “OR” statements like you would in English (will result in “silent error”):

```
1 filter(penguins, year == 2007 | 2008)
```

This does not give a direct error, but does not evaluate the logic the way you want.

```
      species    island bill_len bill_dep flipper_len body_mass    sex year
1      Adelie Torgersen     39.1     18.7          181      3750   male 2007
2      Adelie Torgersen     39.5     17.4          186      3800 female 2007
3      Adelie Torgersen     40.3     18.0          195      3250 female 2007
4      Adelie Torgersen       NA       NA           NA        NA   <NA> 2007
5      Adelie Torgersen     36.7     19.3          193      3450 female 2007
6      Adelie Torgersen     39.3     20.6          190      3650   male 2007
7      Adelie Torgersen     38.9     17.8          181      3625 female 2007
8      Adelie Torgersen     39.2     19.6          195      4675   male 2007
9      Adelie Torgersen     34.1     18.1          193      3475   <NA> 2007
10     Adelie Torgersen     42.0     20.2          190      4250   <NA> 2007
11     Adelie Torgersen     37.8     17.1          186      3300   <NA> 2007
```

distinct()

Keeps only unique/distinct rows from a data frame

First argument: data frame, next arguments: variables from the data frame for which a distinct combination is wanted

```
1 distinct(penguins, species, island)
```

```
    species    island
1    Adelie Torgersen
2    Adelie    Biscoe
3    Adelie     Dream
4    Gentoo    Biscoe
5 Chinstrap     Dream
```

`count()`

Similar to `distinct()`, but rather than showing you unique instances it counts the number of occurrences

First argument: data frame, next arguments: variables whose unique combinations are counted

```
1 count(penguins, species, island)
```

```
    species    island   n
1    Adelie    Biscoe  44
2    Adelie     Dream  56
3    Adelie Torgersen  52
4 Chinstrap     Dream  68
5    Gentoo    Biscoe 124
```

- Optionally, you can arrange the results in descending order using `sort = TRUE` argument.

`slice()`

Allows you to select, remove, and duplicate rows based on their (integer) location. E.g., rows 1-5, rows 20-last row, etc.

First argument: data frame, second argument: integers of the row location.

- **positive** integers indicate you want to keep rows, **negative** integers indicate you want to remove rows

```
1 slice(penguins, 2:4)
```

```
  species    island bill_len bill_dep flipper_len body_mass    sex year
1  Adelie Torgersen     39.5     17.4          186      3800 female 2007
2  Adelie Torgersen     40.3     18.0          195      3250 female 2007
3  Adelie Torgersen       NA       NA           NA        NA   <NA> 2007
```

`slice()`

Allows you to select, remove, and duplicate rows based on their (integer) location. E.g., rows 1-5, rows 20-last row, etc.

First argument: data frame, second argument: integers of the row location.

- **positive** integers indicate you want to keep rows, **negative** integers indicate you want to remove rows

```
1 slice(penguins, -(2:4))
```

	species	island	bill_len	bill_dep	flipper_len	body_mass	sex	year
1	Adelie	Torgersen	39.1	18.7	181	3750	male	2007
2	Adelie	Torgersen	36.7	19.3	193	3450	female	2007
3	Adelie	Torgersen	39.3	20.6	190	3650	male	2007
4	Adelie	Torgersen	38.9	17.8	181	3625	female	2007
5	Adelie	Torgersen	39.2	19.6	195	4675	male	2007
6	Adelie	Torgersen	34.1	18.1	193	3475	<NA>	2007
7	Adelie	Torgersen	42.0	20.2	190	4250	<NA>	2007
8	Adelie	Torgersen	37.8	17.1	186	3300	<NA>	2007
9	Adelie	Torgersen	37.8	17.3	180	3700	<NA>	2007
10	Adelie	Torgersen	41.1	17.6	182	3200	female	2007
11	Adelie	Torgersen	38.6	21.2	191	3800	male	2007

2) Functions that operate on COLUMNS

mutate()

Creates new or modifies existing variables in the object

First argument: data frame, second argument: name-value pair

```
1 penguins_mutated <- mutate(penguins, ratio = flipper_len / body_mass)
```

	species	island	bill_len	bill_dep	flipper_len	body_mass	sex	year	ratio
1	Adelie	Torgersen	39.1	18.7	181	3750	male	2007	0.04826667
2	Adelie	Torgersen	39.5	17.4	186	3800	female	2007	0.04894737
3	Adelie	Torgersen	40.3	18.0	195	3250	female	2007	0.06000000
4	Adelie	Torgersen	NA	NA	NA	NA	<NA>	2007	NA
5	Adelie	Torgersen	36.7	19.3	193	3450	female	2007	0.05594203
6	Adelie	Torgersen	39.3	20.6	190	3650	male	2007	0.05205479

mutate()

Creates new or modifies existing variables in the object

First argument: data frame, second argument: name-value pair

```
penguins_mutated <- mutate(penguins, ratio = flipper_len / body_mass)
penguins_mutated_2 <- mutate(penguins, body_mass_kg = body_mass / 1000)
```

```
  species    island bill_len bill_dep flipper_len body_mass    sex year body_mass_kg
1  Adelie Torgersen     39.1     18.7         181      3750   male 2007         3.75
2  Adelie Torgersen     39.5     17.4         186      3800 female 2007         3.80
3  Adelie Torgersen     40.3     18.0         195      3250 female 2007         3.25
4  Adelie Torgersen       NA       NA          NA        NA   <NA> 2007           NA
5  Adelie Torgersen     36.7     19.3         193      3450 female 2007         3.45
6  Adelie Torgersen     39.3     20.6         190      3650   male 2007         3.65
```

mutate()

Creates new or modifies existing variables in the object

First argument: data frame, second argument: name-value pair

```
penguins_mutated <- mutate(penguins, ratio = flipper_len / body_mass)
penguins_mutated_2 <- mutate(penguins, body_mass_kg = body_mass / 1000)
penguins_mutated_3 <- mutate(penguins, size = ifelse(body_mass > 4000, "Large", "Small"))
```

	species	island	bill_len	bill_dep	flipper_len	body_mass	sex	year	size
1	Adelie	Torgersen	39.1	18.7	181	3750	male	2007	Small
2	Adelie	Torgersen	39.5	17.4	186	3800	female	2007	Small
3	Adelie	Torgersen	40.3	18.0	195	3250	female	2007	Small
4	Adelie	Torgersen	NA	NA	NA	NA	<NA>	2007	<NA>
5	Adelie	Torgersen	36.7	19.3	193	3450	female	2007	Small
6	Adelie	Torgersen	39.3	20.6	190	3650	male	2007	Small

select()

Select (and optionally rename) variables in a data frame

First argument: data frame, next arguments: names of columns to keep or remove

```
1 select(penguins, species, bill_len, bill_dep)
```

```
      species bill_len bill_dep
1      Adelie     39.1     18.7
2      Adelie     39.5     17.4
3      Adelie     40.3     18.0
4      Adelie       NA       NA
5      Adelie     36.7     19.3
6      Adelie     39.3     20.6
7      Adelie     38.9     17.8
8      Adelie     39.2     19.6
9      Adelie     34.1     18.1
10     Adelie     42.0     20.2
11     Adelie     37.8     17.1
12     Adelie     37.8     17.3
13     Adelie     41.1     17.6
14     Adelie     38.6     21.2
15     Adelie     34.6     21.1
```

`select()`

Select/remove variables in a data frame

First argument: data frame, next arguments: names of columns to keep or remove

- Useful functions that can be used as arguments of `select()`: `starts_with()`, `ends_with()`, and `contains()`

```
1  select(penguins, starts_with("bill"))
```

```
    bill_len bill_dep
1       39.1     18.7
2       39.5     17.4
3       40.3     18.0
4         NA       NA
5       36.7     19.3
6       39.3     20.6
7       38.9     17.8
8       39.2     19.6
9       34.1     18.1
```

`select()`

Select/remove variables (columns) in a data frame

First argument: data frame, next arguments: names of columns to keep or remove

- Useful functions that can be used as arguments of `select()`: `starts_with()`, `ends_with()`, and `contains()`

```
1 select(penguins, contains("_"))
```

- As before, putting `-` in front of the argument will remove the specified columns

rename()

Rename variables in a data frame

First argument: data frame, next arguments: “new_name” = old_name, “new_name_2” = old_name_2, …

```
1 rename(penguins, "bill_len_mm" = bill_len)
```

```
     species    island bill_len_mm bill_dep flipper_len body_mass    sex year
1     Adelie Torgersen        39.1     18.7         181      3750   male 2007
2     Adelie Torgersen        39.5     17.4         186      3800 female 2007
3     Adelie Torgersen        40.3     18.0         195      3250 female 2007
4     Adelie Torgersen          NA       NA          NA        NA   <NA> 2007
5     Adelie Torgersen        36.7     19.3         193      3450 female 2007
6     Adelie Torgersen        39.3     20.6         190      3650   male 2007
7     Adelie Torgersen        38.9     17.8         181      3625 female 2007
8     Adelie Torgersen        39.2     19.6         195      4675   male 2007
9     Adelie Torgersen        34.1     18.1         193      3475   <NA> 2007
```

3) Grouping

Group & summarize data

Function `group_by()` *silently* creates grouping of data by a specified variable

This means you will see only `# Groups` added to the data frame header, nothing will change with data itself

```
1 group_by(penguins, species)
```

```
# A tibble: 344 × 8
# Groups:   species [3]
   species island    bill_len bill_dep flipper_len body_mass sex     year
   <fct>   <fct>        <dbl>    <dbl>       <int>     <int> <fct>  <int>
 1 Adelie  Torgersen     39.1     18.7         181      3750 male    2007
 2 Adelie  Torgersen     39.5     17.4         186      3800 female  2007
 3 Adelie  Torgersen     40.3     18           195      3250 female  2007
 4 Adelie  Torgersen     NA       NA            NA        NA <NA>    2007
 5 Adelie  Torgersen     36.7     19.3         193      3450 female  2007
 6 Adelie  Torgersen     39.3     20.6         190      3650 male    2007
 7 Adelie  Torgersen     38.9     17.8         181      3625 female  2007
 8 Adelie  Torgersen     39.2     19.6         195      4675 male    2007
 9 Adelie  Torgersen     34.1     18.1         193      3475 <NA>    2007
10 Adelie  Torgersen     42       20.2         190      4250 <NA>    2007
# ℹ 334 more rows
```

Group & summarize data

Function `summarise()` then collapses many rows into single summary value *per group*, the summary statistic can be mean, median, quartiles, etc.

If data is not grouped, the function will be used on complete data.

- Let's first demonstrate `summarise()` on non-grouped data:

```
summarise(penguins, avg_len = mean(flipper_len))
```

```
  avg_len
1      NA
```

Group & summarize data

Function `summarise()` then collapses many rows into single summary value *per group*, the summary statistic can be mean, median, quartiles, etc.

If data is not grouped, the function will be used on complete data.

- That did not work, R does not know how to deal with `NA` values when applying summary functions

- Ignoring `NA` values can be specified using `na.rm = TRUE` argument:

```
1  summarise(penguins, avg_len = mean(flipper_len, na.rm = TRUE))
```

```
   avg_len
1 200.9152
```

Group & summarize data

`group_by()` and `summarise()` usually work in tandem, where data is first grouped, and then summarized by group.

- Let's pass `penguins` grouped by `species` to the `summarise()` function

```
1 summarise(group_by(penguins, species),
2           avg_len = mean(flipper_len, na.rm = TRUE))
```

```
# A tibble: 3 × 2
  species   avg_len
  <fct>       <dbl>
1 Adelie       190.
2 Chinstrap    196.
3 Gentoo       217.
```

Code readability

Our code chunks will slowly become longer and *more nested*, or we will encounter *naming issues*. Examples:

```
# A nested code chunk
select(
  mutate(
    filter(
      penguins,
      year >= 2008
    ),
    size = ifelse(body_mass > 4000, "large", "small")
  ),
  species, year, size
)
```

```
# So many similar names!
penguins_1 <- filter(penguins, year >= 2008)
penguins_2 <- mutate(penguins_1, size = ifelse(body_mass > 4000, "large", "small"))
penguins_3 <- select(penguins_2, species, year, size)
```

Solution? **Data pipelines**

4) Pipelines

Data pipelines

Data pipelines are a way of changing multiple operations into a clear, readable sequence using the pipe operator: `%>%` (tidyverse) or `|>` (base R).

A shortcut for writing the pipe operator is `ctrl` + `shift` + `M`.

- Let's demonstrate this on the previous example:

```
summarise(group_by(penguins, species),
          avg_len = mean(flipper_len, na.rm = TRUE))
```

- Same example with pipeline approach:

```
penguins %>%
  filter(is.na(flipper_len) == FALSE)
```

Data pipelines

Data pipelines are a way of changing multiple operations into a clear, readable sequence using the pipe operator: `%>%` (tidyverse) or `|>` (base R).

A shortcut for writing the pipe operator is `ctrl` + `shift` + `M`.

- Let's demonstrate this on the previous example:

```
summarise(group_by(penguins, species),
          avg_len = mean(flipper_len, na.rm = TRUE))
```

- Same example with pipeline approach:

```
penguins %>%
  filter(is.na(flipper_len) == FALSE) %>%
  group_by(species, sex)
```

Data pipelines

Data pipelines are a way of changing multiple operations into a clear, readable sequence using the pipe operator: `%>%` (tidyverse) or `|>` (base R).

A shortcut for writing the pipe operator is `ctrl` + `shift` + `M`.

- Let's demonstrate this on the previous example:

```
summarise(group_by(penguins, species),
          avg_len = mean(flipper_len, na.rm = TRUE))
```

- Same example with pipeline approach:

```
penguins %>%
  filter(is.na(flipper_len) == FALSE) %>%
  group_by(species, sex) %>%
  summarise(avg_len = mean(flipper_len))
```

Data pipelines

- Let's try to get medians with interquartile ranges *per penguin species* and *per penguin sex*

```
1  penguins %>%
2    filter(is.na(flipper_len) == FALSE & is.na(sex) == FALSE)
```

Data pipelines

- Let's try to get medians with interquartile ranges *per penguin species* and *per penguin sex*

```
1 penguins %>%
2   filter(is.na(flipper_len) == FALSE & is.na(sex) == FALSE) %>%
3   group_by(species, sex)
```

Data pipelines

- Let's try to get medians with interquartile ranges *per penguin species* and *per penguin sex*

```
penguins %>%
  filter(is.na(flipper_len) == FALSE & is.na(sex) == FALSE) %>%
  group_by(species, sex) %>%
  summarise(median_flipper = median(flipper_len),
            q25 = quantile(flipper_len, 0.25),
            q75 = quantile(flipper_len, 0.75))
```

```
# A tibble: 6 × 5
# Groups:   species [3]
  species   sex    median_flipper   q25   q75
  <fct>     <fct>           <dbl> <dbl> <dbl>
1 Adelie    female            188   185   191
2 Adelie    male              193   189   197
3 Chinstrap female            192   187.  196.
4 Chinstrap male              200.  196   203
5 Gentoo    female            212   210   215
6 Gentoo    male              221   218   225
```

Important functions and concepts, let's revise briefly before moving to next topics

Recap

Key `dplyr` functions (cheat sheet):

- `filter()` - choose rows based on logic operations
- `select()` - choose columns based on column names
- `mutate()` - create new columns or modify existing ones using calculations or functions
- `group_by()` + `summarise()` - aggregate by group to create summary statistics
- `%>%` - chain steps into a pipeline

Next up

Transformed data still needs the right *shape* to work well with `ggplot2`, `dplyr`, and other common R functions.

Part 4 - Data tidying